The Systems Behind Modern AI

Why and how does hardware designed for 3D games power modern AI

Sam Ly

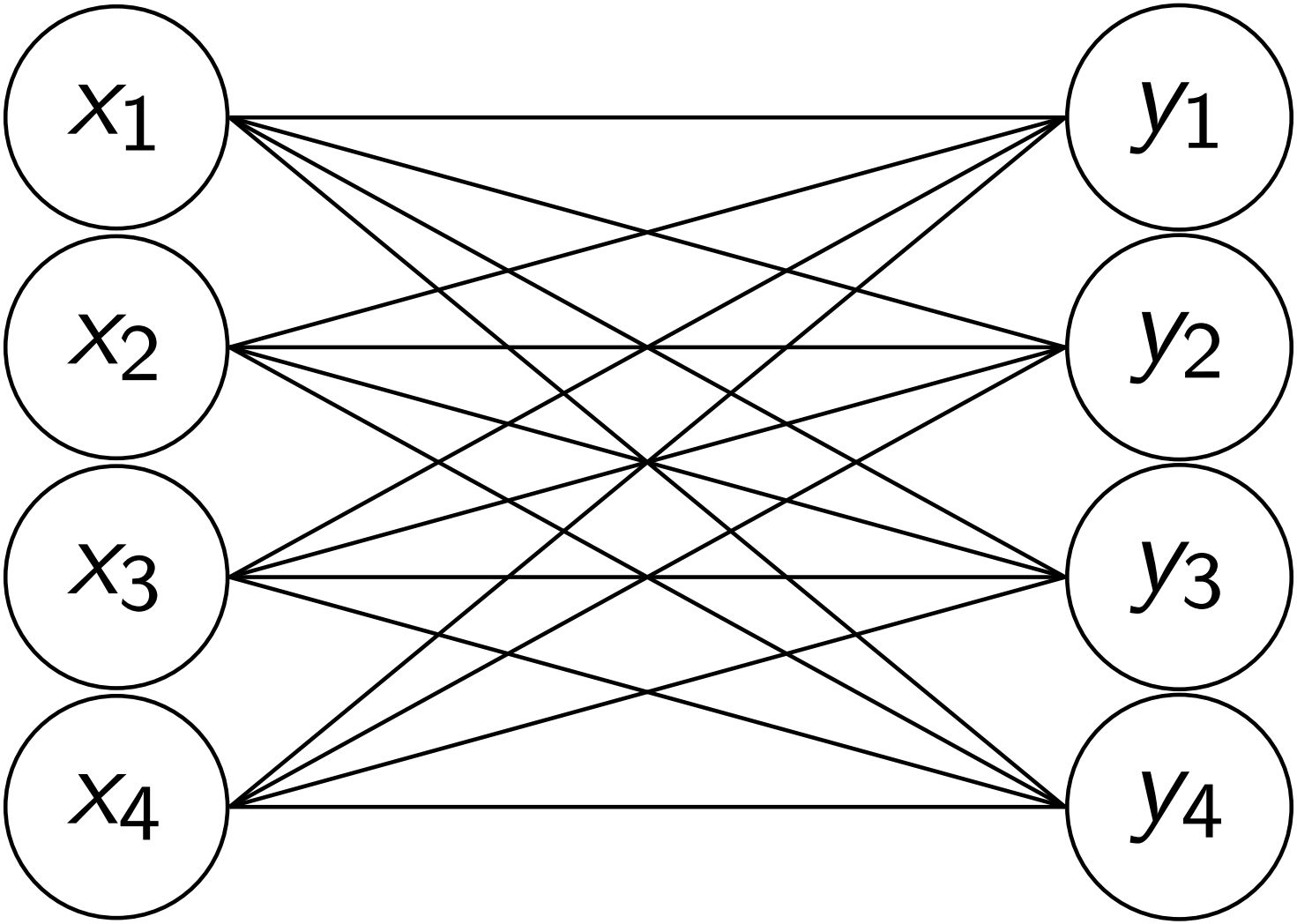
Cal Poly Pomona

Spring 2026

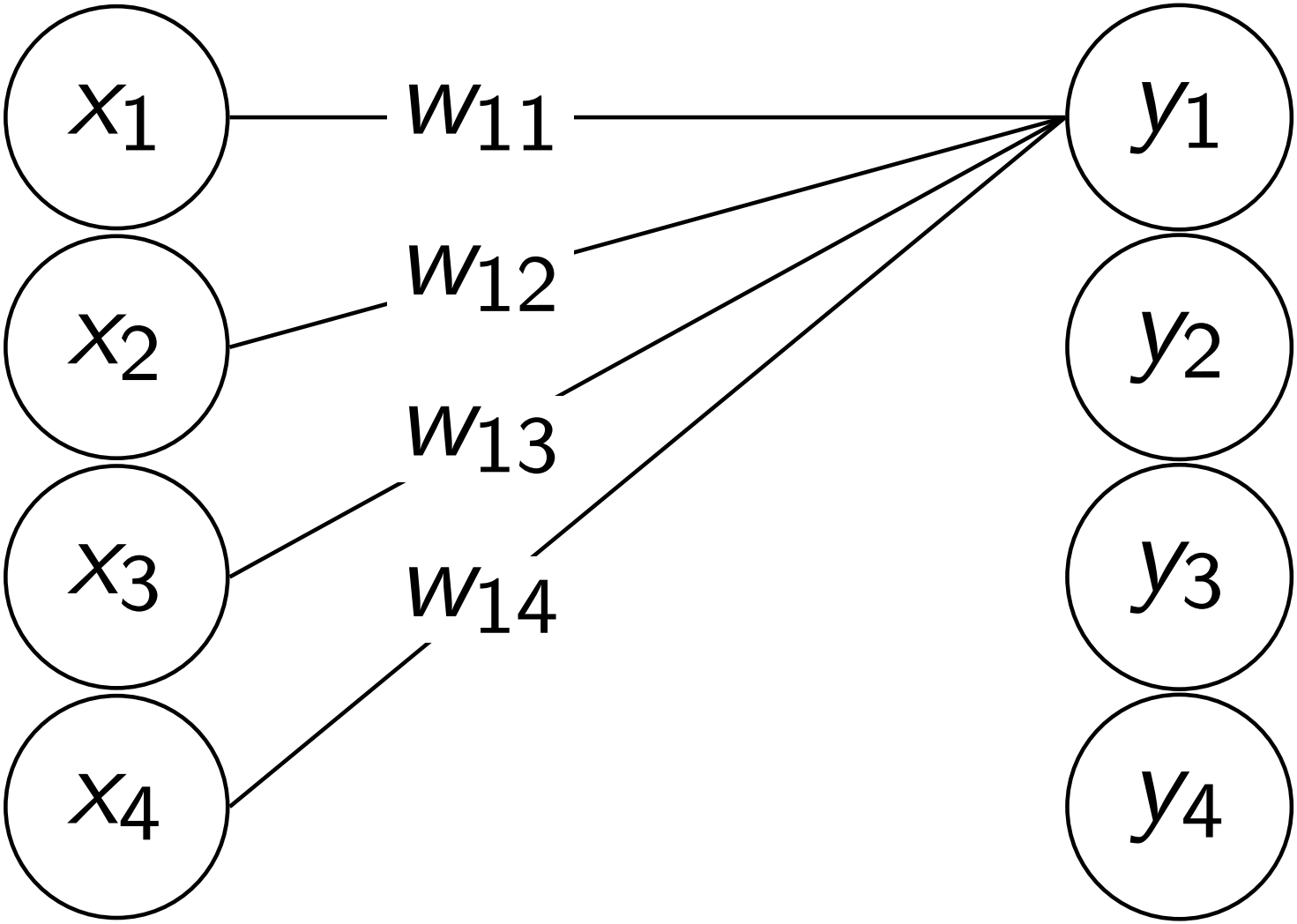
Table of Contents

1. Neural Networks and the Matrix Multiplication
2. Why GPUs?
3. Optimizing **Ge**neral Matrix Multiplication (GEMM)

Basic Neural Net (MLP/FFN)

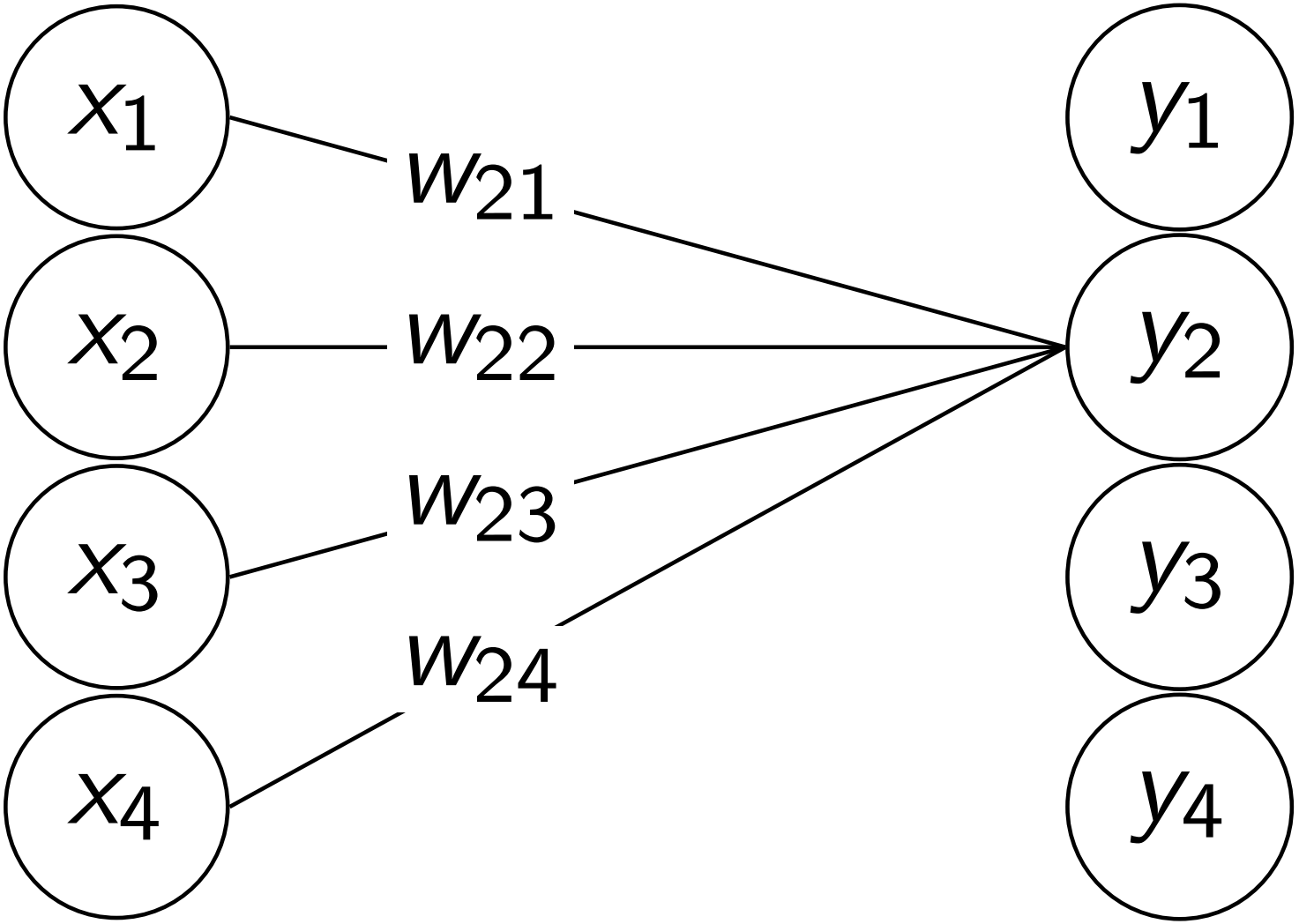


Basic Neural Net (MLP/FFN)



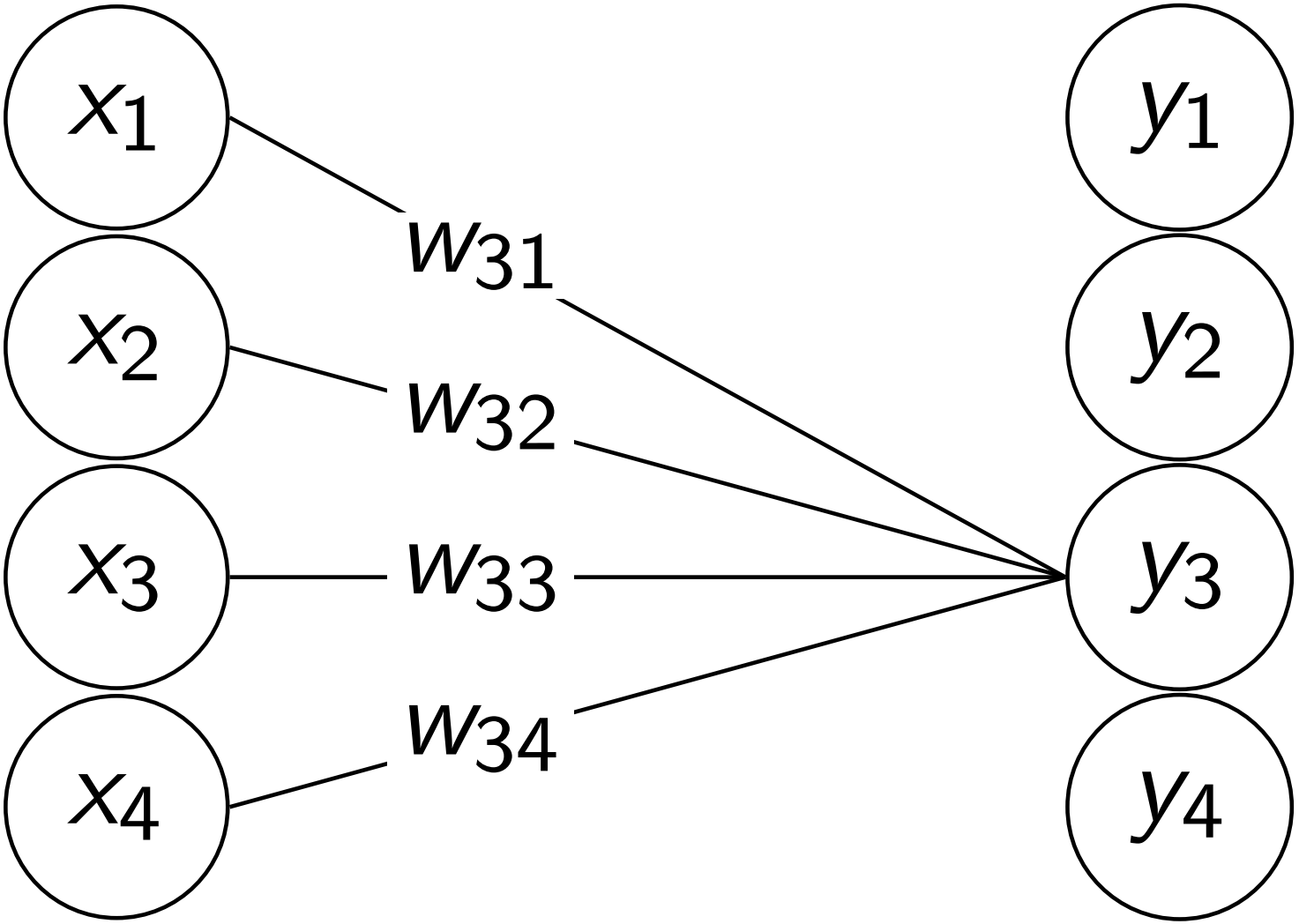
$$y_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14}$$

Basic Neural Net (MLP/FFN)



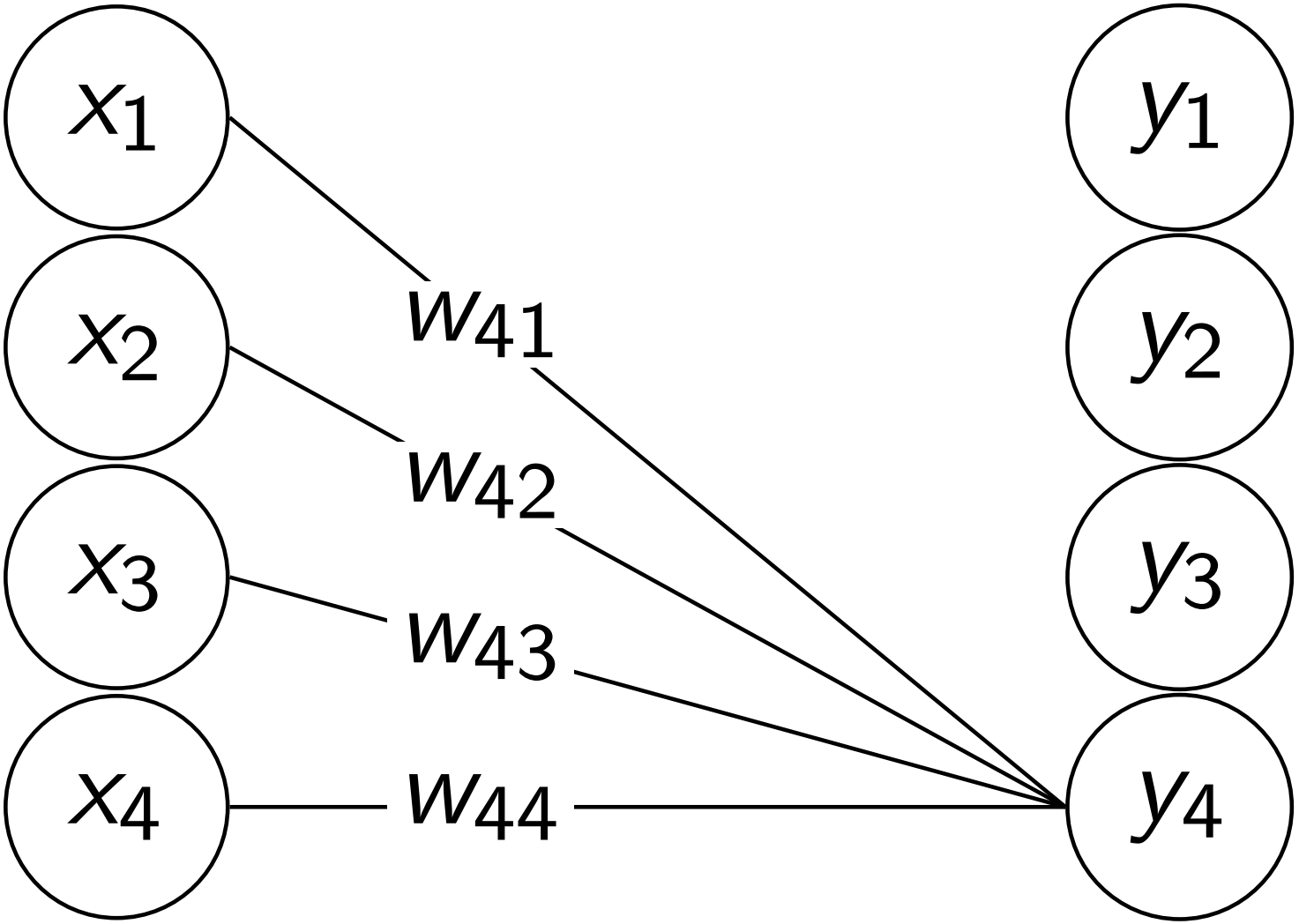
$$y_2 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24}$$

Basic Neural Net (MLP/FFN)



$$y_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34}$$

Basic Neural Net (MLP/FFN)



$$y_4 = x_1 w_{41} + x_2 w_{42} + x_3 w_{43} + x_4 w_{44}$$

Linear Algebra Interpretation

Here, notice that for each y_i, we are taking a weighted sum of the x elements. That is,

$$y_i = x_1 w_{i1} + x_2 w_{i2} + x_3 w_{i3} + x_4 w_{i4}.$$

Arranging the y_i into a vector, we get

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \\ w_{41} & w_{42} & w_{43} & w_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}.$$

Linear Algebra Interpretation

We can generalize the number of x's and the number of y's.

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

We say, n is the *input dimension* and m is the *output dimension*.

Computing with multiple x vectors.

Suppose we have multiple x vectors, for which we want to pass through our neural network. We can stack the multiple x vectors in the **batch**, which we compute with a matrix multiplication.

$$\begin{bmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(B)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ y_m^{(1)} & y_m^{(2)} & \cdots & y_m^{(B)} \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(B)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(B)} \end{bmatrix}.$$

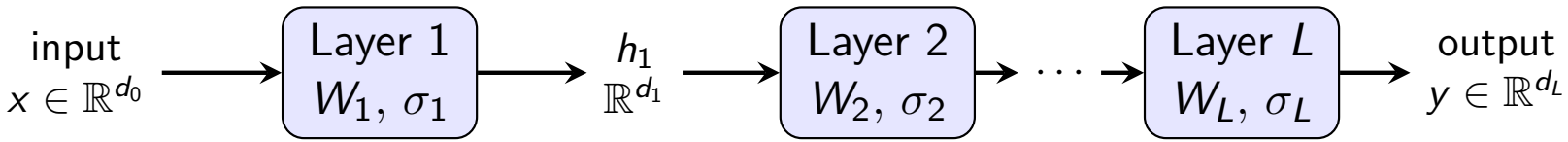
We write this succinctly as $Y = WX$, where $Y \in \mathbb{R}^{m \times B}$, $X \in \mathbb{R}^{n \times B}$, and $W \in \mathbb{R}^{m \times n}$.

Yippee!

We have successfully mapped the batched computation of a **linear layer** a matrix multiplication!

Composing Layers

We can view W as a function that takes in an n-dimensional vector and maps it to an m-dimensional vector. $W : \mathbb{R}^n \to \mathbb{R}^m$. We create deep neural networks by *composing* these transformations, along with **nonlinearities** σ_i.

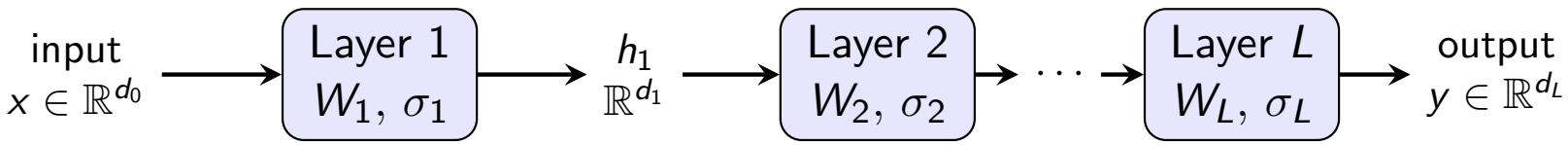


$$h_0 = x, \qquad h_\ell = \sigma_\ell(W_\ell h_{\ell-1}), \qquad y = h_L$$

Remark

Notice that this figure doesn't show the *batch* dimension. However, if you work through the dimension mappings, the *batch* dimension B will be unaffected by this composition.

Limitations for Neural Network Architecture



$$h_0 = x, \qquad h_\ell = \sigma_\ell(W_\ell h_{\ell-1}), \qquad y = h_L$$

Architectures such as **Transformers** and **CNNs** have connectivity patterns more complex than a simple composition of layers. Nevertheless, linear transformations remain a fundamental computational primitive in neural network architectures.

A fundamental building block

A linear transformation may seem like a narrow operation to study, but modern neural networks perform it repeatedly and at enormous scale. Its efficiency therefore has an outsized effect on training and inference performance. The rest of this presentation explores the hardware and techniques that make these operations efficient.

Takeaways

1. A linear layer applied to a batch of inputs can be expressed as a single matrix multiplication, $Y = WX$.
2. Deep networks compose these linear transformations with nonlinearities. More complex architectures change how layers are connected, but linear transformations remain a fundamental building block.
3. **Matrix multiplication therefore captures a large and reusable part of neural network computation.**

Why is this representation useful?

It separates what the network computes from how the hardware executes it. Instead of optimizing every architecture from scratch, we can focus on one common computational primitive. Next, we will examine how GPUs execute matrix multiplications efficiently.

Table of Contents

1. Neural Networks and the Matrix Multiplication
2. Why GPUs?
3. Optimizing **Ge**neral Matrix Multiplication (GEMM)

Demand for 3D graphics

GPU's came about due to the increase in computational power needed to render 3D games in real time. This requires *millions* of FLOP's per second. CPU's could not keep up.
Rendering graphics has a nice property that we can take advantage of: **parallelism**.

Parallelism

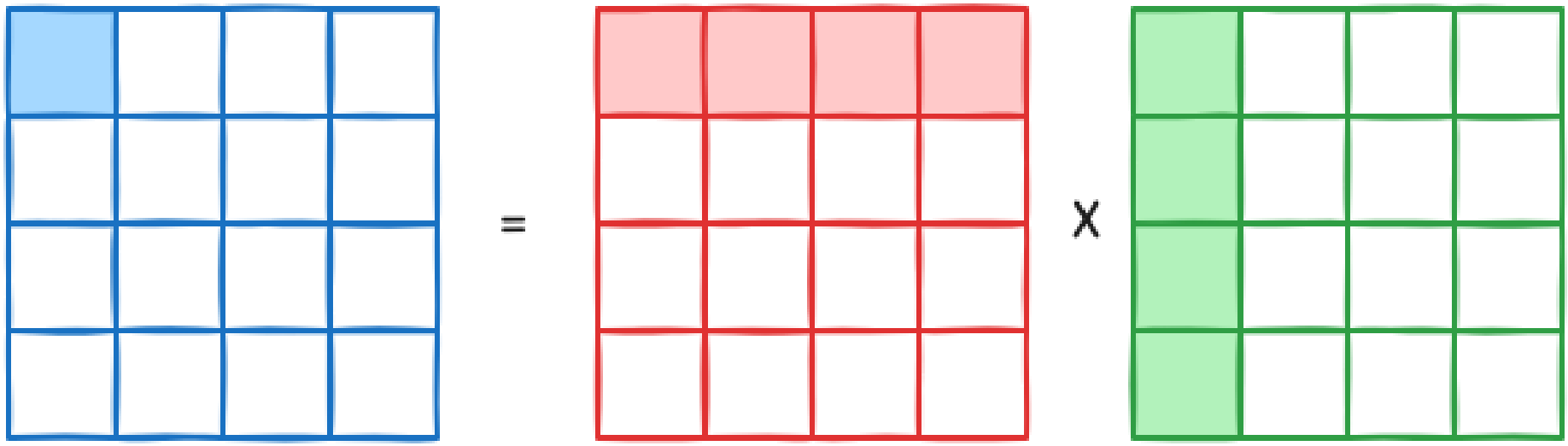
Each pixel on the screen doesn't really care any of the other pixels.
Furthermore, the computation that is required for each pixel isn't that complex either.
The real difficulty in rendering graphics is the sheer volume of computations necessary.
GPU's were designed to take advantage of these properties of rendering 3D graphics.

History of GPUs

- 1999: GeForce 256
- 2004: Brook Programming, more complex "rendering" pipelines, GP-GPU
- 2006: GeForce 8800, Compute Unified Device Architecture (CUDA)
- 2008: Fermi, General Purpose Scientific Compute becomes first class, L1/L2 caches
- 2010-2016: incremental improvements to power efficiency, programming models, and multi-GPU systems.
- 2017: Volta, AI-specialization: tensor cores designed around deep learning primitives
- 2022: Hopper, Transformer Engine, further specializing into AI
- 2026: Since 2022, we have only really seen incremental improvements.

Matrix Multiplication is Parallel

Let A and B be two 4×4 matrices. The matrix multiple $C = AB$ results in another 4×4 matrix.



Each element of the output is independent. The results of C_{11} does not effect the results of C_{12}, or any other element.

Remark

If matrix multiplication is a **parallel** task, and GPU's were designed for the **parallel** task of rendering graphics, can we repurpose GPU's to do matrix multiplication?

CPU implementation

The operations are very simple, but the number of operations grow on the order of $\mathcal{O}(n^3)$, where n is the dimensions of the matrices.

```
for (std::size_t i = 0; i < n; ++i) {
  for (std::size_t j = 0; j < n; ++j) {
    double sum = 0.0;
    for (std::size_t k = 0; k < n; ++k) {
      sum += a[i * n + k] * b[k * n + j];
    }
    c[i * n + j] = sum;
  }
}
```

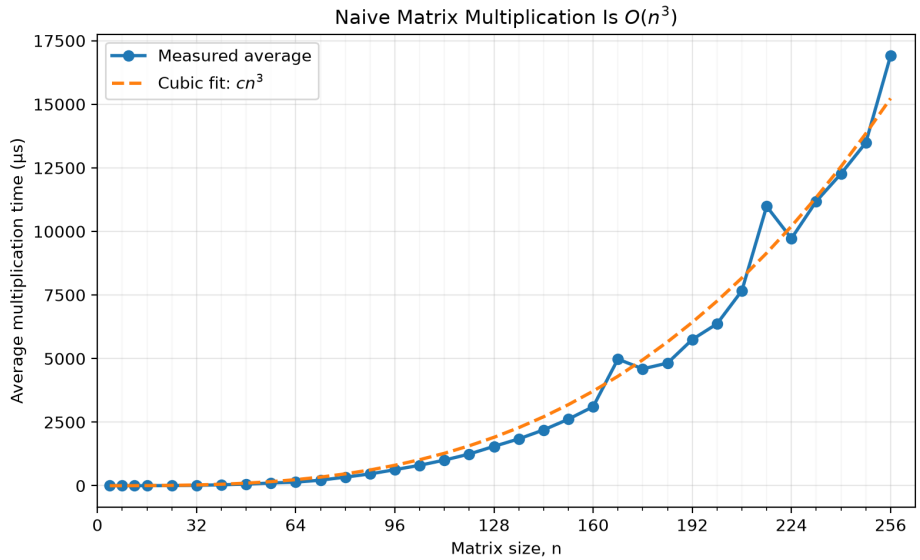


Table of Contents

1 Neural Networks and the Matrix Multiplication

2 Why GPUs?

3 Optimizing **Ge**neral Matrix Multiplication (GEMM)

Naive

Arithmetic Intensity, throughput, and the Roofline Model